```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
from google.colab import files

# ========================================
# STEP 1: DEFINE THE PROBLEM
# Medical Image Segmentation using GWO
# ========================================

# Upload MRI / CT image natively in Google Colab
print("Please upload your medical image:")
uploaded = files.upload()

if not uploaded:
    print("No image selected!")
    # Fallback: create a dummy image if nothing is uploaded so the notebook can still run
    print("Creating a synthetic image for demonstration...")
    image_path = "synthetic_mri.png"
    dummy_img = np.zeros((256, 256), dtype=np.uint8)
    cv2.circle(dummy_img, (128, 128), 60, 200, -1)
    cv2.circle(dummy_img, (128, 128), 30, 100, -1)
    cv2.imwrite(image_path, dummy_img)
else:
    image_path = list(uploaded.keys())[0]

# Load image in grayscale
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error loading image!")
    exit()

# Preprocessing: Remove noise
processed_image = cv2.GaussianBlur(
    image, (5, 5), 0
)

# ========================================
# STEP 2: INITIALIZE PARAMETERS
# ========================================

N = 20             # Number of wolves
MaxIter = 30       # Maximum iterations
D = 2              # Number of thresholds

lower_bound = 0
upper_bound = 255

# Create histogram
histogram = cv2.calcHist(
    [processed_image], [0], None, [256], [0, 256]
).flatten()

histogram = histogram / histogram.sum()

# Calculate cumulative probability and mean
omega = np.cumsum(histogram)
mu = np.cumsum(histogram * np.arange(256))

total_mean = mu[-1]


# ========================================
# STEP 3: FITNESS FUNCTION
# Multi-level Otsu fitness
# ========================================

def fitness_function(wolf):

    thresholds = np.sort(
        np.clip(np.round(wolf).astype(int), 0, 255)
    )

    t1, t2 = thresholds

    # Class probabilities
    w0 = omega[t1]
    w1 = omega[t2] - omega[t1]
    w2 = omega[255] - omega[t2]
```

```
    w2 = omega[255] - omega[t2]

    # Avoid empty classes
    if min(w0, w1, w2) <= 1e-10:
        return 0

    # Class means
    m0 = mu[t1] / w0
    m1 = (mu[t2] - mu[t1]) / w1
    m2 = (mu[255] - mu[t2]) / w2

    # Between-class variance
    variance = (
        w0 * (m0 - total_mean) ** 2
        + w1 * (m1 - total_mean) ** 2
        + w2 * (m2 - total_mean) ** 2
    )

    return variance


# ========================================
# STEP 4: INITIALIZE POPULATION
# ========================================

wolves = np.random.uniform(
    lower_bound, upper_bound, (N, D)
)

# Evaluate initial fitness
fitness = np.array([
    fitness_function(wolf) for wolf in wolves
])

# Identify Alpha, Beta and Delta
rank = np.argsort(fitness)[::-1]

alpha = wolves[rank[0]].copy()
beta = wolves[rank[1]].copy()
delta = wolves[rank[2]].copy()

alpha_score = fitness[rank[0]]

convergence = []

print("\nGrey Wolf Optimizer Started")
print("-------------------------------")


# ========================================
# STEP 5 & 6: UPDATE POSITIONS AND ITERATE
# ========================================

for t in range(MaxIter):

    # Adaptive parameter
    a = 2 - (2 * t / MaxIter)

    for i in range(N):

        X = wolves[i].copy()

        # Random coefficients
        r1 = np.random.random(D)
        r2 = np.random.random(D)

        A1 = 2 * a * r1 - a
        C1 = 2 * r2

        D_alpha = np.abs(C1 * alpha - X)
        X1 = alpha - A1 * D_alpha

        r1 = np.random.random(D)
        r2 = np.random.random(D)

        A2 = 2 * a * r1 - a
        C2 = 2 * r2

        D_beta = np.abs(C2 * beta - X)
        X2 = beta - A2 * D_beta

        r1 = np.random.random(D)
        r2 = np.random.random(D)
```

```
        A3 = 2 * a * r1 - a
        C3 = 2 * r2

        D_delta = np.abs(C3 * delta - X)
        X3 = delta - A3 * D_delta

        # Update wolf position
        new_position = (X1 + X2 + X3) / 3

        # Keep position within valid limits
        wolves[i] = np.clip(
            new_position,
            lower_bound,
            upper_bound
        )

    # Evaluate updated population
    fitness = np.array([
        fitness_function(wolf) for wolf in wolves
    ])

    # Rank wolves
    rank = np.argsort(fitness)[::-1]

    # Update Alpha, Beta and Delta
    alpha = wolves[rank[0]].copy()
    beta = wolves[rank[1]].copy()
    delta = wolves[rank[2]].copy()

    alpha_score = fitness[rank[0]]

    convergence.append(alpha_score)

    print(
        f"Iteration {t + 1}/{MaxIter} "
        f"| Best Fitness: {alpha_score:.4f}"
    )


# ========================================
# STEP 7: OUTPUT THE BEST SOLUTION
# ========================================

best_thresholds = np.sort(
    np.round(alpha).astype(int)
)

t1, t2 = best_thresholds

print("\n--------------------------------")
print("OPTIMIZATION COMPLETED")
print("--------------------------------")

print("Alpha (Best Solution):", alpha)
print("Beta (Second Best):", beta)
print("Delta (Third Best):", delta)

print("Optimal Threshold 1:", t1)
print("Optimal Threshold 2:", t2)

print("Best Fitness Value:", alpha_score)


# ========================================
# STEP 8: MEDICAL IMAGE SEGMENTATION
# ========================================

# Divide image into three regions
segmented_image = np.zeros_like(processed_image)

segmented_image[processed_image <= t1] = 0

segmented_image[
    (processed_image > t1) &
    (processed_image <= t2)
] = 1

segmented_image[processed_image > t2] = 2

# Convert labels into visible grayscale values
segmented_display = (
    segmented_image * 127
```

```
).astype(np.uint8)

# Create colored segmentation
colored_segmentation = plt.get_cmap("jet")(
    segmented_image / 2
)[:, :, :3]

# Create overlay
original_rgb = cv2.cvtColor(
    processed_image, cv2.COLOR_GRAY2RGB
)

overlay = (
    0.6 * original_rgb / 255.0
    + 0.4 * colored_segmentation
)

overlay = np.clip(overlay, 0, 1)


# ========================================
# SAVE OUTPUT IMAGES
# ========================================

output_folder = os.path.join(
    os.getcwd(),
    "GWO_Results"
)

os.makedirs(output_folder, exist_ok=True)

cv2.imwrite(
    os.path.join(output_folder, "Preprocessed.png"),
    processed_image
)

cv2.imwrite(
    os.path.join(output_folder, "Segmented.png"),
    segmented_display
)

plt.imsave(
    os.path.join(output_folder, "Colored_Segmentation.png"),
    colored_segmentation
)

plt.imsave(
    os.path.join(output_folder, "Overlay.png"),
    overlay
)


# ========================================
# DISPLAY RESULTS
# ========================================

plt.figure(figsize=(15, 9))

plt.subplot(2, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original MRI Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(processed_image, cmap="gray")
plt.title("Preprocessed Image")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(segmented_image, cmap="jet", vmin=0, vmax=2)
plt.title("GWO Segmented Image")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(overlay)
plt.title("Segmentation Overlay")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.plot(convergence)
plt.title("GWO Convergence Curve")
plt.xlabel("Iteration")
plt.ylabel("Best Fitness")
```

```
plt.ylabel("Best Fitness")
plt.grid(True)

plt.subplot(2, 3, 6)
plt.hist(
    processed_image.ravel(),
    bins=256,
    range=(0, 256),
    color="gray"
)

plt.axvline(t1, linestyle="--", label=f"T1 = {t1}")
plt.axvline(t2, linestyle="--", label=f"T2 = {t2}")

plt.title("Optimal Thresholds")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.legend()

plt.tight_layout()

plt.savefig(
    os.path.join(output_folder, "GWO_Results.png")
)

plt.show()

print("\nAll results saved in:")
print(output_folder)
```

```
Please upload your medical image:
```
Choose Files bis.jpg
bis.jpg(image/jpeg) - 83761 bytes, last modified: 11/6/2026 - 100% done
```
Saving bis.jpg to bis.jpg

Grey Wolf Optimizer Started
--------------------------------
Iteration 1/30 | Best Fitness: 1982.5989
Iteration 2/30 | Best Fitness: 2018.5456
Iteration 3/30 | Best Fitness: 2000.6140
Iteration 4/30 | Best Fitness: 1929.4124
Iteration 5/30 | Best Fitness: 2009.5772
Iteration 6/30 | Best Fitness: 2007.8468
Iteration 7/30 | Best Fitness: 2021.2171
Iteration 8/30 | Best Fitness: 2007.9045
Iteration 9/30 | Best Fitness: 2023.6003
Iteration 10/30 | Best Fitness: 2022.5884
Iteration 11/30 | Best Fitness: 2026.3934
Iteration 12/30 | Best Fitness: 2027.1803
Iteration 13/30 | Best Fitness: 2026.2333
Iteration 14/30 | Best Fitness: 2023.2607
Iteration 15/30 | Best Fitness: 2027.0392
Iteration 16/30 | Best Fitness: 2027.1718
Iteration 17/30 | Best Fitness: 2026.8930
Iteration 18/30 | Best Fitness: 2026.9078
Iteration 19/30 | Best Fitness: 2025.8186
Iteration 20/30 | Best Fitness: 2026.3196
Iteration 21/30 | Best Fitness: 2025.9670
Iteration 22/30 | Best Fitness: 2026.4451
Iteration 23/30 | Best Fitness: 2026.9342
Iteration 24/30 | Best Fitness: 2026.8754
Iteration 25/30 | Best Fitness: 2027.0803
Iteration 26/30 | Best Fitness: 2027.0814
Iteration 27/30 | Best Fitness: 2027.1692
Iteration 28/30 | Best Fitness: 2027.1692
Iteration 29/30 | Best Fitness: 2027.1718
Iteration 30/30 | Best Fitness: 2027.1718

--------------------------------
OPTIMIZATION COMPLETED
--------------------------------
Alpha (Best Solution): [112.82898102  37.5288285 ]
Beta (Second Best): [112.97578688  37.60191842]
Delta (Third Best): [112.89301909  37.81080666]
Optimal Threshold 1: 38
Optimal Threshold 2: 113
Best Fitness Value: 2027.171776364639
```











